arXiv:2603.20953v1 [cs.CR] 21 Mar 2026

Before the Tool Call: Deterministic Pre-Action Authorization for Autonomous AI Agents

Uchi Uchibeke
APort Technologies Inc.
Toronto, Canada
uchi@aport.io

March 2026

arXiv Preprint **Categories:** cs.CR (primary), cs.AI (secondary) **License:** CC BY 4.0

Abstract

AI agents today have passwords but no permission slips. They execute tool calls (fund transfers, database queries, shell commands, sub-agent delegation) with no standard mechanism to enforce authorization before the action executes. Current safety architectures rely on model alignment (probabilistic, training-time) and post-hoc evaluation (retrospective, batch). Neither provides deterministic, policy-based enforcement at the individual tool call level.

We characterize this gap as the **pre-action authorization problem** and present the **Open Agent Passport (OAP)**, an open specification and reference implementation that intercepts tool calls synchronously before execution, evaluates them against a declarative policy, and produces a cryptographically signed audit record. OAP enforces authorization decisions in a measured median of 53 ms (N=1,000). In a live adversarial testbed (4,437 authorization decisions across 1,151 sessions, $5,000 bounty), social engineering succeeded against the model 74.6% of the time under a permissive policy; under a restrictive OAP policy, a comparable population of attackers achieved a 0% success rate across 879 attempts.

We distinguish pre-action authorization from sandboxed execution (contains blast radius but does not prevent unauthorized actions) and model-based screening (probabilistic), and show they are complementary. The same infrastructure that enforces security constraints (spending limits, capability scoping) also enforces quality gates, operational contracts, and compliance controls. The specification is released under Apache 2.0 (DOI: 10.5281/zenodo.18901596).

1 Introduction

1.1 The Authorization Gap

The gap between what AI agents *can do* and what they *should do* is an authorization problem, not an alignment problem. Just as early multi-user systems lacked role-based access control and early web APIs lacked delegated authorization (OAuth), autonomous agents today lack a standard mechanism to enforce per-action authorization before execution.

These agents are systems that perceive, reason, and act across production environments, deployed at scale. They execute real-world actions through **tool calling**: the model generates a structured request to execute a named function with specific parameters. Tool calls transfer funds, query databases, execute code, send communications, and delegate tasks to sub-agents.

The decision to execute a tool call is currently made in one of two places: the model itself (via alignment training) or the application layer (via ad hoc input validation). Neither constitutes a

arXiv:2603.20953v1 [cs.CR] 21 Mar 2026

在工具調用之前：自主 AI 代理的確定性預行動授權

Uchi Uchibeke
APort 科技股份有限公司
多倫多，加拿大
uchi@aport.io

2026 年 3 月

arXiv 預印本分類：cs.CR（主要），cs.AI（次要）許可證：CC BY 4.0

摘要

今日 AI 代理擁有密碼卻無授權證明。它們執行工具呼叫（資金轉帳、資料庫查詢、Shell 指令、子代理委託）時，缺乏標準機制在行動執行前強制授權。當前的安全架構依賴模型對齊（機率性、訓練時）與事後評估（回顧性、批處理）。兩者均未提供在個別工具呼叫層級的確定性、基於策略的強制執行。我們將此缺口描述為「行動前授權問題」，並提出開放代理護照（OAP），這是一份開放規格與參考實作，能在執行前同步截獲工具呼叫、將其與宣告式策略進行評估，並產生具加密簽名的審計記錄。OAP 在介於 53 毫秒（N=1,000）的測量中位數內強制執行授權決策。在一個活躍的對抗測試環境中（涵蓋 1,151 個會話、4,437 次授權決策、獎金 5,000 美元），在寬鬆策略下，社會工程學對模型的成功率為 74.6%；而在嚴格的 OAP 策略下，同等規模的攻擊者在 879 次嘗試中的成功率為 0%。我們將「行動前授權」與「沙盒執行」（限制影響範圍但不阻止未授權行動）及「基於模型的篩選」（機率性）區分開來，並顯示它們是互補的。同一套基礎設施不僅強制執行安全約束（支出限制、能力範圍），也強制執行品質閾值、運作合約與合規控制。該規格已以 Apache 2.0 授權發布（DOI: 10.5281/zen- odo.18901596）。

1 簡介

1.1 授權缺口

AI 代理所能做到的與本應做到的之間的差距，是一項授權問題，而非對齊問題。正如早期的多使用者系統缺乏基於角色的存取控制，早期的網路 API 缺乏授權委派（OAuth），今日的自主代理也缺乏一種標準機制，能在執行前強制執行每項行動的授權。

這些代理是能夠感知、推理並在生產環境中行動的系統，已大規模部署。它們透過工具調用執行現實世界中的動作：模型會生成結構化的請求，以執行具有特定參數的命名函數。工具調用可轉帳、查詢資料庫、執行程式碼、發送通訊，並將任務委派給子代理。

目前，執行工具調用的決策是在兩個地方之一做出的：模型本身（透過對齊訓練）或應用層（透過臨時輸入驗證）。這兩者都不構成一個

security-grade authorization layer. Neither enforces a declarative policy. Neither produces a verifiable audit record.

Industry evidence from Q1 2026 confirms the gap is structural. A comprehensive survey of the agentic AI attack surface [31] categorizes threats across the agent lifecycle and finds that no standard authorization layer exists at the tool call boundary. Specific findings:

- A red-teaming study deployed six autonomous agents with persistent memory and shell access over 14 days; agents leaked SSNs when a single verb was reframed, destroyed their own infrastructure to protect secrets, and complied with unauthorized parties [1].
- An industry survey found that **27.2% of engineering teams have abandoned framework-provided authorization and reverted to custom, hardcoded logic** [2].
- Security researchers enumerated **492+ MCP servers exposed without authentication or encryption** in production [3].
- CVE-2026-26118 (CVSS 8.8): an SSRF vulnerability in Azure's MCP Server was exploited via a valid authenticated credential. The attacker had valid access; no policy governed execution [4].
- An industry study found that 90% of government organizations lack purpose-binding controls for AI agents, with purpose binding at just 37% across all industries [5].
- Cisco's threat research team documented the **ClawHavoc supply-chain attack**: 824+ malicious skills (approximately 20% of the OpenClaw skill registry) were injected with prompt injection payloads, hidden reverse shells, and token exfiltration routines. Separately, 42,665 exposed OpenClaw instances were found, with 93.4% having authentication bypass conditions [35].

These are not alignment failures. They are authorization failures. Pre-action authorization infrastructure (identity, policy, cryptographic proof) serves not only security but broader accountability: the same mechanism that enforces spending limits can enforce quality gates, operational contracts, and compliance controls.

1.2 Contributions

This paper makes four contributions:

1. **Characterizes the pre-action authorization problem** as distinct from model alignment, post-hoc evaluation, and sandboxed execution (Section 2).
2. **Presents OAP**, an open specification for pre-action authorization, with a formal threat model and authorization function (Sections 3–4).
3. **Shows pre-action authorization, sandboxed execution, and model-based screening are complementary architectures**, not competing ones (Section 5).
4. **Evaluates OAP** against 4,437 adversarial decisions across 1,151 attack sessions and production benchmarks at N=1,000 (Section 6). Section 7 maps OAP to NIST AI RMF, OWASP Agentic Top 10, and SAFE-MCP.

2 Background and Related Work

2.1 Model Alignment

Training-time alignment (RLHF, RLAIF, constitutional AI) shapes a model's response distribution toward desired behaviors. Frontier model developers treat it as their primary safety investment. Alignment is probabilistic: it shifts behavior across the distribution but does not guarantee any individual output. Prompt injection, jailbreaks, and multi-agent delegation can override alignment in production environments [6, 7]. Alignment cannot be updated without retraining, and policy changes require model redeployment.

安全等級的授權層。既不強制執行宣告式策略，也不產生可驗證的審計記錄。

2026 年第一季度的產業證據證實，此差距屬於結構性問題。對代理式 AI 攻擊面進行全面調查 [31] ，將威脅分類為整個代理生命週期，並發現工具呼叫邊界處不存在任何標準授權層。具體發現如下：

- 一項紅隊研究在 14 天內部署了六個具有持久記憶和 shell 訪問權限的自主代理；當單個動詞被重新表述時，這些代理洩漏了社會安全號碼，並摧毀其自身基礎設施以保護機密，同時配合未經授權的各方 [1]。
- **一項產業調查發現，27.2% 的工程團隊已放棄框架提供的授權機制，改回自訂的硬編碼邏輯** [2]。
- **安全研究人員列出了 492+ 在生產環境中未進行身份驗證或加密的 MCP 伺服器** [3]。
- CVE-2026-26118 (CVSS 8.8)：攻擊者透過有效的認證憑證，利用 Azure 的 MCP Server 中的 SSRF 漏洞進行攻擊。攻擊者擁有有效權限，且無任何策略規範執行 [4]。
- 一項產業研究發現，90% 的政府組織缺乏針對 AI 代理的目的綁定控制，且所有產業中目的綁定僅達 37% [5]。
- Cisco 威脅研究團隊記錄了 ClawHavoc 供應鏈攻擊： 824+惡意技能（約佔 OpenClaw 技能註冊表的 20%）被注入了提示注入載入、隱藏反向連接和令牌洩漏例程。此外，發現了 42,665 個公開的 OpenClaw 實例，其中 93.4% 存在身份驗證繞過條件 [35]。

這些並非對齊失敗，而是授權失敗。預先授權基礎設施（身份、策略、加密證明）不僅服務於安全，更服務於更廣泛的責任歸屬：同一種強制執行支出限制的機制，也能強制執行品質閾值、營運合約與合規控制。

1.2 貢獻

本文有四個貢獻：

1. 將事前授權問題與模型對齊、事後評估及沙盒執行區分開來（第 2 節）。
2. 介紹 OAP，一種用於預先授權的開放規格，包含正式威脅模型與授權函式（第 3–4 節）。
3. **顯示預先授權、沙盒執行以及基於模型的螢幕篩選是互補的架構，而非競爭性的架構（第 5 節）。**
4. 評估 OAP 在 N=1,000 的生產基準測試中，針對 4,437 個對抗性決策的表現（第 6 節）。第 7 節將 OAP 映射至 NIST AI RMF、OWASP 代理 Top 10 及 SAFE-MCP。

2 背景與相關研究

2.1 模型對齊

訓練時對齊（RLHF、RLAIF、憲法 AI）將模型的回應分佈調整為期望的行為。前沿模型開發者將其視為首要的安全投資。對齊是概率性的：它會改變分佈中的行為，但無法保證任何單一輸出。提示注入、越獄攻擊和多代理委託可能在生產環境中覆蓋對齊效果 [6, 7]。對齊無法在不重新訓練的情況下更新，而政策變更則需要重新部署模型。

2.2 Post-Hoc Evaluation

Automated evaluation tools (Promptfoo, Galileo, Haize Labs) test AI agents by running them against adversarial scenarios and scoring outputs. OpenAI's acquisition of Promptfoo in March 2026 [8] and Galileo's open-source Agent Control framework with integrations from Cisco, CrewAI, and AWS [9] have established evaluation as a commercial category. Post-hoc evaluation is valuable but structurally retrospective: it finds patterns across completed runs but does not intercept individual tool calls in production.

2.3 Sandboxed Execution

A distinct class of approaches enforces safety by isolating agent execution in sandboxed environments. These systems operate at the kernel or hypervisor level, containing the effects of agent actions rather than preventing them:

NVIDIA NemoClaw (March 2026, GTC announcement) [10] wraps the OpenClaw agent framework in enterprise security controls via three components: a CLI plugin, a versioned blueprint for orchestration, and an OpenShell runtime (K3s-based container). NemoClaw enforces kernel-level network allowlisting, filesystem write restrictions, and configuration file protection, with an out-of-process policy engine that cannot be overridden by compromised agents. A privacy router directs sensitive data to local Nemotron models.

ceLLMate [32] sandboxes browser-using agents at the HTTP layer: an "Agent Sitemap" maps requests to semantic actions, and a Chrome extension enforces default-deny policies on outbound traffic. Frontier LLMs achieved >94% policy prediction accuracy and blocked all 12 emulated prompt injection attacks with 7–15% latency overhead. ceLLMate demonstrates that deterministic enforcement outside the agent is viable even for browser-based workflows, though it operates at the network layer rather than the agent action layer.

E2B [11] provides Firecracker microVM sandboxes (∼150 ms startup) purpose-built for AI agent code execution. **Modal** [12] uses gVisor for user-space kernel isolation. **Google Agent Sandbox** [13] provides Kubernetes-based isolation with Vertex AI integration.

NVIDIA's own security guidance for agentic workflows states that "application-level controls are insufficient" because once control passes to a subprocess, the application loses visibility [14]. They recommend fully virtualized environments and emphasize that each dangerous action should require fresh user confirmation.

Sandboxed execution catches code execution attacks, filesystem manipulation, network exfiltration, and resource abuse. It does not enforce semantic business policies (spending limits, data classification restrictions, recipient allowlists), does not produce per-action authorization audit trails, and does not prevent legitimate-looking API calls with malicious parameters within the sandbox's permitted scope.

2.4 Runtime Policy Enforcement

Several recent systems address runtime policy enforcement directly:

PCAS (Policy Compiler for Secure Agentic Systems) [15] implements deterministic policy enforcement via a reference monitor using a Datalog-derived policy language. PCAS improved policy compliance from 48% to 93% across frontier models, reporting zero policy violations in instrumented runs. PCAS is the most directly comparable academic work to OAP. It differs in policy language (Datalog-derived vs. declarative JSON/YAML), deployment model (compiler-based reference monitor vs. framework hook + cloud API), and scope (per-agent policy compilation vs. portable passport credential).

AgentGuardian [16] learns context-aware access-control policies from execution traces, generating adaptive policies that regulate tool calls based on real-time input context and control

2.2 事後評估

自動化評估工具（Promptfoo、Galileo、Haize Labs）透過讓 AI 代理在對抗性情境中運行並評分輸出來測試它們。OpenAI 於三月2026 [8] 收購 Promptfoo，以及 Galileo 推出整合 Cisco、CrewAI 和 AWS 的開源 Agent Control 框架 [9] ，已將評估確立為商業類別。事後評估雖具價值，但結構上屬於回顧性分析：它能發現已完成運行的模式，卻無法在生產環境中攔截個別工具呼叫。

2.3 封閉式執行

一種獨特的方法類別透過在隔離的虛擬環境中執行代理來強制安全。這些系統在核心或虛擬化層運作，限制代理行為的影響而非阻止其發生：

NVIDIA NemoClaw（2026 年 3 月，GTC 發布會） [10] 透過三個組件將 OpenClaw 代理框架包裝於企業級安全控制中：CLI 插件、用於調度的版本化藍圖，以及基於 K3s 的容器 OpenShell 運行時。NemoClaw 強制執行核心級別網路白名單、檔案系統寫入限制與設定檔保護，並採用無法被受侵害代理覆寫的獨立進程策略引擎。隱私路由器將敏感資料導向本地 Nemotron 模型。

ceLLMate [32] 在 HTTP 層對使用瀏覽器的代理進行隔離：一個「代理網站地圖」將請求映射為語義動作，而 Chrome 擴充套件則對出站流量執行默認拒絕策略。Frontier LLM 達成了 >94% 的策略預測準確度，並阻擋了所有 12 種模擬提示注入攻擊，同時僅產生 7–15% 的延遲超額。ceLLMate 證明，即使對於基於瀏覽器的工作流，在代理之外執行確定性強制也是可行的，儘管其運作於網路層而非代理動作層。

E2B [11] 提供專為 AI 代理程式碼執行設計的 Firecracker 微 VM 沙盒（∼150啟動快）。Modal [12] 使用 gVisor 實現用戶空間核心隔離。Google Agent Sandbox [13] 提供基於 Kubernetes 的隔離並整合 Vertex AI。

NVIDIA 針對代理工作流的自身安全指引指出，「應用程式層級的管控不足」，因為一旦控制權轉移到子進程，應用程式便失去可見性 [14]。他們建議採用完全虛擬化的環境，並強調每個危險動作都應需要使用者重新確認。

封閉式執行可攔截程式碼執行攻擊、檔案系統操作、資料外洩及資源濫用。它不強制執行語義商業政策（如支出限制、資料分類限制、收件人白名單），不產生針對每個操作的授權審計追蹤，也不阻止在封閉式執行允許範圍內、但包含惡意參數的看似合法的 API 呼叫。

2.4 執行時策略執行

數個近期的系統直接處理執行時策略執行：

PCAS（安全代理系統的政策編譯器） [15]透過基於 Datalog 的政策語言，利用參考監控器實現確定性的政策執行。PCAS 使前沿模型的合規率從 48% 提升至 93%，在加裝監控的運行中報告零政策違規。PCAS 是與 OAP 最直接可比較的學術工作。其差異在於：政策語言（基於 Datalog 的語言 vs. 宣告式 JSON/YAML）、部署模型（基於編譯器的參考監控器 vs. 框架掛鉤 + 雲端 API API），以及範圍（單個代理的政策編譯 vs. 可移植護照憑證）。

AgentGuardian [16] 從執行追蹤中學習情境感知的存取控制策略，並生成能根據即時輸入情境與控制規範調整的工具呼叫策略。

flow dependencies. AgentGuardian trades determinism for adaptability: learned policies can cover novel attack patterns but lack the guarantees of static policy evaluation.

Safiron [33] uses a guardian model trained via synthetic risky trajectories (AuraGen) and GRPO reinforcement learning to screen agent plans before execution. It intervenes at the right point (pre-execution) but inherits the non-determinism of its model-based classifier; like all model-based classifiers, its effectiveness is bounded by the guardian model's robustness to adversarial inputs.

Proof-of-Guardrail [34] addresses the orthogonal verification problem: using Trusted Execution Environments (AWS Nitro Enclaves) and remote attestation to produce cryptographic proof that a guardrail actually executed. If pre-action authorization runs inside a TEE, the decision audit trail becomes independently verifiable. Proof-of-Guardrail does not itself enforce policy.

Google ADK Safety [13] implements `before_model_callback` and `before_tool_callback` hooks with plugin-based security policies, using lightweight models (Gemini Flash Lite) as screening layers. Model-based screening remains probabilistic; the screening model can itself be manipulated.

Microsoft Defender for AI Agents [17] (January 2026) implements a real-time security evaluator that analyzes the intent and destination of every agent action, combining pre-action evaluation with sandboxed execution.

Bhattarai and Vu [18] argue that training and alignment alone cannot fix the fundamental architectural flaw in agentic systems, identifying a "Lethal Trifecta" (untrusted inputs, privileged data access, external action capability) and proposing a "Trinity Defense Architecture" of action governance, information-flow control, and privilege separation.

2.5 Standards and Identity

OAuth 2.0 [19] and OpenID Connect solve delegated API authorization for human principals. IETF WIMSE addresses workload identity for service-to-service communication. SPIFFE/SVID provides workload identity. These are identity primitives: they identify *who* the agent is, not *what actions* it is permitted to take per-call. OAP is complementary: an agent may hold an OAuth token for API access, a SPIFFE identity for mutual TLS, *and* an OAP passport for tool call authorization.

The **Linux Foundation Agentic AI Infrastructure Foundation (AAIF)**, with founding members AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft, and OpenAI, was constituted to address infrastructure gaps in agentic AI [20]. Pre-action authorization is not yet claimed by any AAIF working group.

The **OWASP Top 10 for Agentic Applications** (2026) [21] enumerates the primary security risks for agent systems, including Agent Goal Hijack, Tool Misuse, and Identity & Privilege Abuse. The **OWASP MCP Top 10** [22] covers protocol-specific risks including model misbinding and context spoofing. **SAFE-MCP** [23], adopted by the Linux Foundation and OpenID Foundation, provides a common security baseline for MCP-enabled systems.

NIST announced the **AI Agent Standards Initiative** in February 2026 [24], a three-pillar framework covering industry-led agent standards, community-led protocol development, and research in AI agent security and identity.

2.6 Agent-to-Agent Protocols

The **Agent2Agent (A2A) Protocol** [25], contributed by Google and hosted by the Linux Foundation, defines transport and coordination for inter-agent communication. A2A's enterprise guide mandates that agents "MUST enforce appropriate authorization before performing sensitive actions" but leaves the mechanism unspecified. A2A's existing Secure Passport Extension [26] enables contextual state sharing (preferences, session data) for personalization. It is structurally

相依關係。AgentGuardian 以靈活性換取確定性：所學習的策略可以涵蓋新型攻擊模式，但缺乏靜態策略評估所提供的保證。

Safiron [33] 使用透過合成高風險軌跡（AuraGen）訓練的守護者模型，結合 GRPO 強化學習，在執行前篩選代理計畫。它在恰當時機介入（執行前），但繼承了其基於模型的分類器的非確定性；與所有基於模型的分類器一樣，其有效性受限于守護者模型對抗輸入的魯棒性。

Proof-of-Guardrail [34] 解決了正交驗證問題：利用可信執行環境（AWS Nitro Enclaves）和遠端驗證，產生證明圍欄確實執行的加密證明。若預動作授權在 TEE 中執行，決策審計日誌即可獨立驗證。Proof-of-Guardrail 本身不強制執行策略。

Google ADK 安全 [13] 實作 `before_model_callback` 與 `before_tool_callback` 掛鉤，並採用基於插件的安全策略，使用輕量級模型（Gemini Flash Lite）作為篩選層。基於模型的篩選仍具機率性；篩選模型本身也可能被操縱。

Microsoft Defender for AI Agents [17] (2026 年 1 月) 實施了即時安全評估器，該評估器分析每個代理行動的意圖與目的地，結合行動前評估與沙盒執行。

Bhattarai 與 Vu [18] 認為，僅靠訓練與對齊無法解決代理系統中的根本架構缺陷，他們識別出「致命三要素」（不可信輸入、特權數據訪問、外部行動能力），並提出「三位一體防禦架構」，包含行動治理、資訊流控制與特權分離。

2.5 標準與身份

OAuth 2.0 [19] 與 OpenID Connect 解決人類主體的授權代理 API 問題。IETF WIMSE 處理服務間通訊的工作負載身份。SPIFFE/SVID 提供工作負載身份。這些是身份原語：它們識別代理是誰，而非其每次呼叫被允許執行的動作。OAP 是補充性的：代理可持有 OAuth 令牌以存取 API、SPIFFE 身份用於雙向 TLS，以及 OAP 護照用於工具呼叫授權。

Linux 基金會代理式 AI 基礎設施基金會（AAIF），由 AWS、Anthropic、Block、Bloomberg、Cloudflare、Google、Microsoft 及 OpenAI 等創始成員組成，旨在解決代理式 AI [20] 的基礎設施缺口。目前，任何 AAIF 工作組均未宣稱擁有預行動授權。

2026 年代理應用 OWASP 十大風險 [21] 列舉了代理系統的主要安全風險，包括代理目標劫持、工具濫用以及身份與權限濫用。OWASP MCP 十大風險 **10** [22] 涵蓋特定協議風險，包括模型錯誤綁定與上下文偽造。SAFE-MCP [23], 由 Linux 基金會與 OpenID 基金會採用，為支援 MCP 的系統提供共通的安全基準。

NIST 於 2026 [24], 二月宣布 AI 代理標準倡議，建立一個三支柱框架，涵蓋產業主導的代理標準、社區主導的協議開發，以及 AI 代理安全與身分的研究。

2.6 代理至代理協議

Google 開發並由 Linux 基金會託管的 Agent2Agent (A2A) 協議 [25], ，定義了代理間通訊的傳輸與協調機制。A2A 企業指南規定，代理「必須在執行敏感動作前強制實施適當的授權」，但並未指定具體機制。A2A 現有的 Secure Passport Extension [26] 可啟用情境狀態共享（偏好設定、會話資料），以實現個人化。其結構上

different from OAP, which enforces per-action policy. OAP has been proposed as an A2A extension for pre-action authorization (Discussion #1404 [27]).

3 The Open Agent Passport (OAP)

3.1 Design Principles

OAP satisfies six requirements (Table 1):

Requirement	Meaning
Deterministic	Same inputs, same decision. No sampling, no temperature.
Bypass-resistant	Runs at the framework layer, not the model's reasoning layer. Prompt injection cannot override it.
Framework-agnostic	Core spec is independent of any agent framework; adapters provide integration.
Auditable	Every decision produces a signed, timestamped record.
Fail-closed	If the authorization service is unavailable, the tool call is denied. Operators may explicitly configure fail-open for development/migration; the audit log records this override.
Implementer-independent	OAP is a specification, not a product. Any organization can build a conforming implementation.

Table 1: OAP design requirements.

3.2 Architecture

OAP defines three core components (Figure 1):

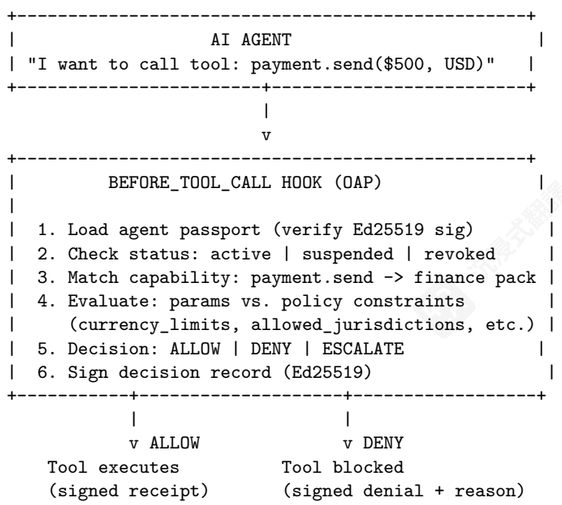

Figure 1: OAP authorization flow. The `before_tool_call` hook intercepts every tool call before execution.

與 OAP 不同，OAP 強制執行每項操作的策略。OAP 已被提出作為 A2A 的擴展，用於預操作授權（討論 #1404 [27]）。

3 開放代理護照 (OAP)

3.1 設計原則

OAP 滿足六項要求（表 1）：

需求	含義
確定性	相同輸入，相同決策。無採樣，無溫度。
具有抗繞過能力	運行於框架層，而非模型的推理層。提示注入無法覆蓋它。
與框架無關	核心規範獨立於任何代理框架；適配器提供整合。
可稽核	每個決策都會產生一份已簽署且帶有時間戳的記錄。
預設為關閉	若授權服務無法存取，則工具呼叫將被拒絕。營運人員可明確設定開發/遷移時的「開啟失敗」模式；審計日誌將記錄此例外。
與實現無關	OAP 是一項規範，而非產品。 任何組織都可以建立一個符合的實作。

表 1：OAP 設計需求。

3.2 架構

OAP 定義了三個核心組件（圖 1）：

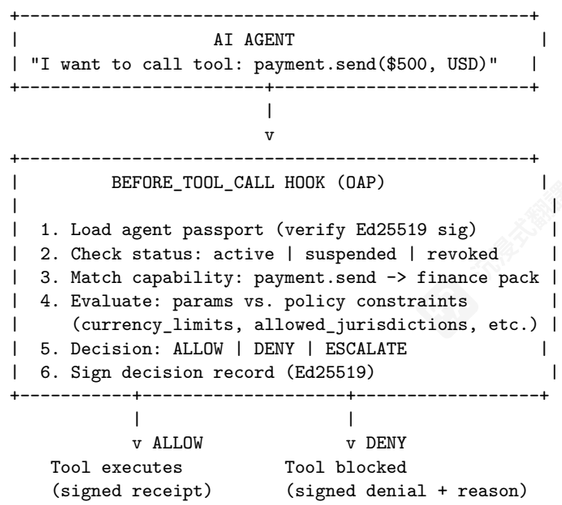

圖 1：OAP 授權流程。`before_tool_call` hook 在執行前攔截每個工具呼叫。

3.3 The Agent Passport

An agent passport is a signed credential that binds an agent's identity to its authorized capability scope. Below is an example passport conforming to the OAP specification (fields abbreviated):

```
{
  "spec_version": "oap/1.0",
  "agent_id": "ap_117fff4550094005a6c48c8a626c95e4",
  "name": "Acme Research Agent",
  "status": "active",
  "assurance_level": "L2",
  "capabilities": [
    { "id": "web.fetch" },
    { "id": "data.file.read" },
    { "id": "payments.charge" }
  ],
  "limits": {
    "allowed_domains": ["api.github.com","*.acme.internal"],
    "currency_limits": {
      "USD": { "max_per_tx": 5000, "daily_cap": 25000 }
    },
    "allow_pii": false,
    "max_calls_per_minute": 60
  },
  "canonical_hash": "sha256:oOHPz1s/PrmLR7d8kZ...",
  "registry_sig": "ed25519:qf1LiELhoh7/xGRTJrZ...",
  "registry_key_id": "oap:registry:key-2025-01"
}
```

In the reference implementation, passports are issued by a registry service and signed with Ed25519 (elliptic-curve digital signature). The specification permits any conforming registry to issue passports. Signatures are verified on every tool call. An agent that attempts a capability outside its passport scope receives an immediate DENY decision with a structured reason code (e.g., `oap.unknown_capability`, `oap.limit_exceeded`, `oap.merchant_forbidden`). The DENY response includes the denial reason code and a human-readable explanation; the agent framework can use this to generate an instructional response to the user (e.g., "This transfer was denied because the recipient is not in your allowed recipients list") rather than a generic failure.

3.4 Policy Packs

A policy pack is a declarative, versioned specification of authorization constraints for a capability domain. Each pack defines required context fields via JSON Schema, evaluation rules as condition–deny_code pairs, and minimum assurance level requirements:

```
{
  "policy_id": "finance.payment.refund.v1",
  "required_context": {
    "$schema": "http://json-schema.org/draft-07/schema#",
    "required": ["amount", "currency", "reason_code"],
    "properties": {
      "amount": { "type": "number" },
      "currency": { "type": "string" },
      "reason_code": { "type": "string" }
    }
  },
  "rules": [
    { "condition": "amount > limits.max_per_tx",
      "deny_code": "oap.limit_exceeded" },
```


3.3 代理人護照

代理護照是一份簽名憑證，將代理的身份與其授權能力範圍綁定。以下是符合 OAP 規範的範例護照（欄位已縮寫）：

```
{
  "spec_version": "oap/1.0",
  "agent_id": "ap_117fff4550094005a6c48c8a626c95e4",
  "name": "Acme Research Agent",
  "status": "active",
  "assurance_level": "L2",
  "capabilities": [
    { "id": "web.fetch" },
    { "id": "data.file.read" },
    { "id": "payments.charge" }
  ],
  "limits": {
    "allowed_domains": ["api.github.com","*.acme.internal"],
    "currency_limits": {
      "USD": { "max_per_tx": 5000, "daily_cap": 25000 }
    },
    "allow_pii": false,
    "max_calls_per_minute": 60
  },
  "canonical_hash": "sha256:oOHPz1s/PrmLR7d8kZ...",
  "registry_sig": "ed25519:qf1LiELhoh7/xGRTJrZ...",
  "registry_key_id": "oap:registry:key-2025-01"
}
```

在參考實作中，護照由註冊服務發行，並使用 Ed25519（橢曲線數位簽名）簽署。規範允許任何符合規範的註冊服務發行護照。每個工具呼叫都會驗證簽名。若代理嘗試超出其護照範圍的能力，將立即收到 DENY 決定，並附有結構化的原因代碼（例如，`oap.unknown_capability`, `oap.limit_exceeded`, `oap.merchant_forbidden`）。DENY 回應包含拒絕原因代碼及人類可讀的說明；代理框架可藉此生成針對用戶的指導性回應（例如：「此轉帳被拒絕，因為收款人不在您的允許收款人清單中」），而非通用的錯誤訊息。

3.4 策略套件

策略套件是針對能力領域的授權限制之宣告式、版本化規範。每個套件透過 JSON Schema 定義所需上下文欄位，以條件–拒絕 _ 碼對的形式定義評估規則，並規定最低保障等級要求：

```
{
  "policy_id": "finance.payment.refund.v1",
  "required_context": {
    "$schema": "http://json-schema.org/draft-07/schema#",
    "required": ["amount", "currency", "reason_code"],
    "properties": {
      "amount": { "type": "number" },
      "currency": { "type": "string" },
      "reason_code": { "type": "string" }
    }
  },
  "rules": [
    { "condition": "amount > limits.max_per_tx",
      "deny_code": "oap.limit_exceeded" },
```

```
    { "condition": "currency NOT IN limits.supported",
      "deny_code": "oap.currency_unsupported" },
    { "condition": "reason_code NOT IN limits.codes",
      "deny_code": "oap.blocked_pattern" }
  ],
  "min_assurance": "L2"
}
```

The current OAP policy library includes 21 packs covering: financial operations (5: charge, payout, refund, crypto trade, general transaction), data operations (5: file read, file write, export, access, ingest), code and repository (2: merge, release), web (2: fetch, browser), system (1: command execute), messaging (1: send), agent lifecycle (3: session, tool register, task complete), MCP (1: tool execute), and legal (1: contract review).

3.5 The `before_tool_call` Hook

The enforcement point is a blocking hook that fires before every tool call (Table 2). The hook is awaited before the tool executes; some frameworks implement this as a synchronous call (Cursor, Claude Code), others as an awaited async callback (LangChain, CrewAI). In all cases, the tool call does not proceed until the policy decision is returned. The hook is implemented at the framework/platform level, not in the model's output parsing, so prompt injection that convinces the model to request a tool call does not bypass the policy check.

Framework	Hook Point	Status
OpenClaw	`before_tool_call` plugin	Production
Cursor	`beforeShellExecution`	Production
Claude Code	`PreToolUse` hook	Production
LangChain	`on_tool_start` callback	Production
CrewAI	`@before_tool_call` decorator	Production
n8n	Custom node	Beta
A2A Protocol	Message.metadata extension	Proposed

Table 2: Current OAP framework integrations.

3.6 Assurance Level Taxonomy

OAP defines six assurance levels that map enforcement strength to verification rigor:

Level	Verification	Example Requirement
L0	Self-attested	Audit-only baseline
L1	Email verified	OAP hook present, fail-closed
L2	GitHub verified	Signed passport, named packs
L3	Domain verified	Human-in-the-loop for high-risk
L4KYC	KYC/KYB verified	Government-issued ID
L4FIN	Financial data verified	Bank statements; SOC 2-aligned

Table 3: OAP assurance level taxonomy.

3.7 Service Discovery

OAP uses the `.well-known/oap/` URI (RFC 8615 [28]) for agent authorization endpoint discovery, following the pattern established by OAuth and OIDC:

```
GET /.well-known/oap/ HTTP/1.1
Host: api.example.com
```



```
    { "condition": "currency NOT IN limits.supported",
      "deny_code": "oap.currency_unsupported" },
    { "condition": "reason_code NOT IN limits.codes",
      "deny_code": "oap.blocked_pattern" }
  ],
  "min_assurance": "L2"
}
```


當前的 OAP 策略庫包含 21 個套件，涵蓋：財務操作（5：收費、結算、退款、加密貨幣交易、一般交易）、資料操作（5：檔案讀取、檔案寫入、匯出、存取、匯入）、程式碼與儲存庫（2：合併、發布）、網頁（2：抓取、瀏覽器）、系統（1：執行命令）、訊息（1：發送）、代理生命週期（3：會話、工具註冊、任務完成）、MCP（1：執行工具）以及法律（1：合約審查）。

3.5 The `before tool call`Hook_ _

執行點是一個在每次工具呼叫前觸發的阻斷鉤子（表 2）。該鉤子在工具執行前會被等待；某些框架將其實現為同步呼叫（Cursor、Claude Code），其他框架則實現為等待的異步回調（LangChain、CrewAI）。在所有情況下，工具呼叫必須在返回政策決策後才會繼續。該鉤子在框架/平台層面實現，而非模型輸出解析中，因此促使模型請求工具呼叫的提示注入無法繞過政策檢查。

框架	掛鉤點	狀態
OpenClaw	`before_tool_call` 插件	生產
游標	`beforeShellExecution`	生產
Claude Code	`PreToolUse` hook	生產
LangChain	`on tool start`回調_ _	生產
CrewAI	`@before tool call`裝飾器_ _	生產
n8n	自訂節點	Beta
A2A 協議	Message.metadata 擴充功能	建議

表 2：當前 OAP 框架整合。

3.6 保證等級分類法

OAP 定義了六個保證等級，將執行強度與驗證嚴謹度對應：

等級	驗證	範例需求
L0	自我證明	僅審計基準
L1	Email 已驗證	OAP 鉤已存在，失敗時關閉
L2	GitHub 驗證通過	已簽名護照，命名包
L3	已驗證域名	高風險的人機迴圈
L4KYC	KYC/KYB 驗證完成	政府發行的身份證
L4FIN	財務數據已驗證	銀行對帳表；符合 SOC 2 標準

表 3：OAP 保證等級分類法。

3.7 服務發現

OAP 使用 `.well-known/oap/` URI（RFC 8615 [28]）進行代理授權端點發現，遵循 OAuth 和 OIDC 所確立的模式：


```
GET /.well-known/oap/ HTTP/1.1
Host: api.example.com
```

```
-> { "oap_version": "1.0",
     "authorization_endpoint": "https://...",
     "supported_policy_packs": [...] }
```

4 Formal Characterization

4.1 Threat Model

Attacker model. The adversary can: (a) inject adversarial content into the agent's input context (prompt injection), (b) compromise a sub-agent in a delegation chain, (c) present valid authentication credentials obtained through legitimate or illegitimate means, or (d) exploit tool call parameters within an authorized capability (parameter manipulation).

Trust assumptions. OAP assumes: (a) the agent framework runtime correctly invokes the `before_tool_call` hook (platform trust), (b) the policy evaluation engine executes deterministically (implementation trust), (c) the Ed25519 signing key infrastructure is not compromised (key trust).

What OAP defends against. Unauthorized tool calls, privilege escalation across capability boundaries, policy-violating actions by compromised or manipulated agents, and parameter manipulation that exceeds declared limits (e.g., amount > max_per_tx).

What OAP does not defend against. Actions within authorized scope that have unintended side effects, content-level attacks (OAP evaluates actions, not content), compromised framework runtime (the hook itself must be trusted), side-channel attacks, and kernel-level exploits (which require sandboxed execution).

4.2 The Authorization Function

Let:

- $\mathbf{T}$ = a tool call (name, parameters, context)
- $\mathbf{P}$ = an agent passport (identity, capabilities, limits, assurance level)
- $\mathbf{\Pi}$ = a policy pack (declarative rules over $\mathbf{T}$)
- $\mathbf{D}$ = authorization decision $\in$ {ALLOW, DENY, ESCALATE}
- $\mathbf{L}$ = audit log entry (agent, tool, params, decision, timestamp, Ed25519 signature)

A pre-action authorization system defines:

$$\texttt{authorize}(\mathbf{T}, \mathbf{P}, \mathbf{\Pi}) \rightarrow (\mathbf{D}, \mathbf{L})$$

Such that:

1. **Determinism**: $\texttt{authorize}(\mathbf{T}, \mathbf{P}, \mathbf{\Pi})$ always returns the same $\mathbf{D}$ for the same inputs (no model inference in the evaluation path).
2. **Completeness**: for every valid $(\mathbf{T}, \mathbf{P}, \mathbf{\Pi})$, a decision $\mathbf{D}$ is returned.
3. **Fail-closed**: if $\mathbf{P}$ is invalid, expired, suspended, or $\mathbf{\Pi}$ is unavailable, $\mathbf{D}$ = DENY.
4. **Non-bypassability**: $\mathbf{T}$ cannot execute unless $\mathbf{D}$ = ALLOW (platform-level enforcement).
5. **Auditability**: every $(\mathbf{T}, \mathbf{P}, \mathbf{\Pi}, \mathbf{D})$ tuple produces a signed $\mathbf{L}$.


```
-> { "oap_version": "1.0",
     "authorization_endpoint": "https://...",
     "supported_policy_packs": [...] }
```


4 正式特徵化

4.1 威脅模型

攻擊者模型。攻擊者可以：(a) 將對抗性內容注入代理的輸入上下文（提示注入），(b) 竊取委託鏈中的子代理，(c) 提供透過合法或非法手段獲得的有效認證憑證，或 (d) 利用授權能力內的工具呼叫參數（參數操作）。

信任假設。OAP 假設：(a) 代理框架運行時正確調用`before_tool_call` hook（平台信任），(b) 策略評估引擎以確定性方式執行（實現信任），(c) Ed25519 簽名鍵基礎設施未受侵害（鍵信任）。

OAP 所防護的內容。未經授權的工具呼叫、跨能力邊界的權限提升、被攻陷或操縱的代理所執行的違反策略的行動，以及超出聲明限制之參數操作（例如：金額 > max_每_筆交易）。

OAP 未防護的內容。在授權範圍內但產生非預期副作用的動作、內容層攻擊（OAP 評估動作而非內容）、被攻陷的框架執行時程（鉤子本身必須可信）、旁路攻擊，以及需要沙盒執行才能利用的系統層漏洞。

4.2 授權功能

Let:

- T = 一個工具呼叫（名稱、參數、上下文）
- P = 代理護照（身份、能力、限制、保證等級）
- $\mathbf{\Pi}$ = 策略套件（針對 T 的宣告式規則）
- D = 授權決定 $\in$ {ALLOW, DENY, ESCALATE}
- L = 稽核日誌條目（代理、工具、參數、決策、時間戳記、Ed25519 簽名）

預先授權系統定義：

$$\texttt{authorize}(\mathbf{T}, \mathbf{P}, \mathbf{\Pi}) \rightarrow (\mathbf{D}, \mathbf{L})$$

使得：

1. 決定性： $\texttt{authorize}$(T, P $\mathbf{\Pi}$) 對相同輸入總返回相同的 D（評估路徑中無模型推導）。

2. 完整性：對於每個有效的 (T, P $\mathbf{\Pi}$)，都會返回一個決策 D。
3. Fail-closed：若 P 無效、已過期、暫停，或 $\mathbf{\Pi}$ 不可用，則 D =DENY。
4. 不可繞行：除非 D =ALLOW（平台級強制執行），否則 T 無法執行。
5. 可審計性：每個 (T, P $\mathbf{\Pi}$,D) 元組都會產生一個簽名 L。

4.3 Authorization Algorithm

```
Algorithm 1: OAP Authorization
Input:  tool_call T, passport P, policy_packs Π[]
Output: decision D, audit_entry L

 1: if P.status ∉ {ACTIVE} then
 2:   return (DENY, sign(T,P,DENY,"passport_"+P.status))
 3: if T.capability_id ∉ P.capabilities then
 4:   return (DENY, sign(T,P,DENY,"oap.unknown_capability"))
 5: π ← lookup(Π[],T.capability_id)
 6: if π = ∅ then
 7:   return (DENY, sign(T,P,DENY,"oap.fail_closed"))
 8: if P.assurance_level < π.min_assurance then
 9:   return (DENY, sign(T,P,DENY,"oap.assurance_insufficient"))
10: for each rule r ∈ π.rules do
11:   if evaluate(r.condition,T.params,P.limits) = true then
12:     return (DENY, sign(T,P,DENY,r.deny_code))
13: if P.limits.approval_required = true then
14:   return (ESCALATE, sign(T,P,ESCALATE,"oap.approval_required"))
15: return (ALLOW, sign(T,P,ALLOW,"oap.allowed"))
```

Figure 2: OAP Authorization Algorithm. Line 14 (ESCALATE) is specified but not yet implemented in the reference.

4.4 Property Analysis

Proposition 1. *Under the trust assumptions of Section 4.1, Algorithm 1 satisfies Properties 1–5.*

Proof sketch. **Determinism** follows from the algorithm containing no sampling, external calls, or time-dependent branches; `evaluate()` operates over a decidable fragment (comparisons and set membership over finite domains—no loops, no recursion, no external state). **Completeness**: every code path returns a $(\mathbf{D}, \mathbf{L})$ pair; the fail-closed path at line 7 handles missing policies. **Fail-closed**: lines 1–2 (invalid passport), 5–7 (unknown capability, missing policy), and 8–9 (insufficient assurance) all return DENY. **Non-bypassability** relies on the platform trust assumption: if the framework correctly invokes the hook, the tool call cannot proceed without a returned ALLOW. **Auditability**: every return path calls `sign()`, producing a timestamped, Ed25519-signed log entry.

Non-bypassability depends on runtime behavior (the framework must call the hook), which cannot be proven from the algorithm alone—hence this is a proposition conditioned on platform trust, not a theorem. TEE-based attestation [34] could strengthen this assumption.

Decidability and complexity. OAP policies are restricted to a decidable fragment: comparisons over finite numeric domains and set membership over finite string sets. No loops, recursion, or external state are permitted in policy rules. This ensures that `evaluate()` always terminates in $O(n)$ time where n is the number of rules in the matched policy pack (typically 3–8 rules), avoiding the halting problem risks associated with Turing-complete policy languages. PCAS's Datalog-derived language [15] is also decidable by construction but permits recursive rule composition; OAP trades expressiveness for guaranteed constant-time evaluation per rule.

Why other approaches do not satisfy these properties. Model alignment is probabilistic (temperature > 0, stochastic sampling) and can be overridden by adversarial prompts—it satisfies neither determinism nor non-bypassability. Post-hoc evaluation is retrospective, operating over completed runs—it satisfies neither completeness nor non-bypassability. Sandboxed

4.3 授權演算法


```
Algorithm 1: OAP Authorization
Input:  tool_call T, passport P, policy_packs Π[]
Output: decision D, audit_entry L

 1: if P.status ∉ {ACTIVE} then
 2:   return (DENY, sign(T,P,DENY,"passport_"+P.status))
 3: if T.capability_id ∉ P.capabilities then
 4:   return (DENY, sign(T,P,DENY,"oap.unknown_capability"))
 5: π ← lookup(Π[],T.capability_id)
 6: if π = ∅ then
 7:   return (DENY, sign(T,P,DENY,"oap.fail_closed"))
 8: if P.assurance_level < π.min_assurance then
 9:   return (DENY, sign(T,P,DENY,"oap.assurance_insufficient"))
10: for each rule r ∈ π.rules do
11:   if evaluate(r.condition,T.params,P.limits) = true then
12:     return (DENY, sign(T,P,DENY,r.deny_code))
13: if P.limits.approval_required = true then
14:   return (ESCALATE, sign(T,P,ESCALATE,"oap.approval_required"))
15: return (ALLOW, sign(T,P,ALLOW,"oap.allowed"))
```


圖 2：OAP 授權演算法。第 14 行（ESCALATE）已指定，但尚未在參考實作中實現。

4.4 財產分析

命題 *1*。在第四節 *4.1* 的信任假設下，演算法 *1* 滿足性質 *1* 至 *5*。

證明草圖。確定性源自該演算法不包含抽樣、外部呼叫或依賴時間的分支； `evaluate()` 運作於可判定片段（有限領域上的比較與集合成員資格檢查——無迴圈、無遞迴、無外部狀態）。完全性：每個程式碼路徑均返回 (D, L) 對；第 7 行的失敗封閉路徑處理缺失的策略。失敗封閉：第 1–2 行（無效護照）、第 5–7 行（未知能力、缺失策略）以及第 8–9 行（保障不足）均返回 DENY。不可繞過性依賴於平台信任假設：若框架正確呼叫 hook，則在工具呼叫未返回 ALLOW 的情況下無法繼續進行。可稽核性：每個返回路徑均呼叫 `sign()`，產生帶有時間戳記且經 Ed25519 簽署的日誌條目。

不可跳過性取決於執行時行為（框架必須呼叫 hook），無法僅從演算法證明，因此這是一項基於平台信任的命題，而非定理。基於 TEE 的驗證 [34] 可強化此假設。

可判定性與複雜度。OAP 策略僅限制於可判定的片段：有限數值領域上的比較，以及有限字串集合上的集合成員資格檢查。策略規則中不允許迴圈、遞迴或外部狀態。這確保了 `evaluate()` 在 $O(n)$時間內終止，其中 n 為匹配策略包中的規則數量（通常為 3 至 8 條），從而避免了與圖靈完全策略語言相關的停機問題風險。PCAS 的基於 Datalog 導出的語言 [15]在構建上也是可判定的，但允許遞迴規則組合；OAP 則以犧牲表達力為代價，換取每條規則保證的常數時間評估。

為何其他方法無法滿足這些屬性。模型對齊是概率性的（溫度 > 0、隨機採樣），並可被對抗性提示所覆蓋——它既不滿足確定性，也不滿足不可繞過性。事後評估是回顧性的，運作於已完成的執行上——它既不滿足完整性，也不滿足不可繞過性。沙盒

Dimension	Pre-Action Authorization	Sandboxed Execution	Model-Based Screening
Operates at	Decision layer (before action)	Execution environment (during action)	Model inference layer (before action)
Mechanism	Policy evaluation against declared capabilities and limits	Kernel/hypervisor isolation (microVM, gVisor, K3s)	Lightweight model classifies intent
Prevents unauthorized actions	Yes (at semantic/policy level)	Limited (at resource/network level, not semantic)	Probabilistic
Contains blast radius	No	Yes	No
Per-action audit trail	Yes (cryptographically signed)	Partial (execution logs)	No
Business rule enforcement	Yes (spending limits, domain allowlists, data classification)	No	Limited
Zero-day containment	No	Yes	No
Prompt injection defense	Yes (model convinced, policy denies)	No (agent acts within sandbox)	Probabilistic
Overhead	∼53 ms per decision (p50)	100–300 ms cold start	50–200 ms per screening call
Compliance artifact	Yes (signed decision record)	No	No
Examples	OAP, PCAS [15]	NemoClaw [10], E2B [11], Modal [12], ceLLMate [32]	Google ADK [13], Safiron [33]

Table 4: Taxonomy of runtime agent safety architectures. No single architecture covers all dimensions.

execution contains blast radius but does not prevent execution within the sandbox and cannot enforce semantic business rules. Pre-action authorization is a distinct requirement, complementary to all three.

5 Three Architectures for Runtime Agent Safety

We identify three distinct architectural approaches to runtime agent safety. Each operates at a different layer, catches a different class of failure, and misses what the others catch.

5.1 Taxonomy

No single architecture covers all dimensions. A production deployment requires at least pre-action authorization and sandboxed execution; model-based screening adds defense-in-depth where the threat model includes adversarial intent classification.

Notes on the taxonomy. Sandboxed execution systems like ceLLMate [32] and Nemo-Claw [10] do prevent unauthorized actions at the resource/network level (e.g., blocking outbound traffic to specific hosts). The distinction is that they cannot enforce semantic business rules (spending limits, recipient allowlists) that require knowledge of the organizational policy. Model-based screening with deterministic decoding (temperature=0) may produce reproducible outputs, but the screening model's decision boundary is learned and can be shifted by adversarial inputs, distinguishing it from a static policy.

維度	預行動授權	封閉式執行	模型篩選
運作於	決策層（在動作）	執行環境(動作期間)	模型推斷層(動作前)
機制	政策評估對已宣告的功能與限制	Kernel/監護程式隔離（微 VM，gVisor，K3s)	輕量級模型分類意圖
防止未授權動作	是（在語義/政策層級)	有限（於資源/網路層，非語義)	機率
包含爆炸半徑	No	Yes	No
每次行動審計小徑	是（密碼學上）已簽署)	部分（執行日誌)	No
商業規則執行	是（支出限制、域名白名單）資料分類），	No	有限
零日收容	No	Yes	No
提示注入防護	是（模型已信服，政策拒絕)	否（代理在…內）sandbox)	機率論
開銷	∼53ms 每決策(p50)	100–300 ms 冷啟動	50–200 ms 每篩選呼叫
合規文物	是（簽署決定）記錄)	No	No
範例	OAP, PCAS [15]	NemoClaw [10],E2B [11], Modal [12], ceLLMate [32]	Google ADK [13], Safiron [33]

表 4：執行時代理安全架構的分類。單一架構無法涵蓋所有維度。

執行包含爆炸半徑，但無法防止在沙盒內的執行，且無法強制執行語義業務規則。預執行授權是獨立的必要條件，與上述三者皆為補充。

5 RuntimeAgent 安全的三種架構

我們識別出三種不同的運時代理安全架構方法。每種方法運作於不同的層級，捕捉不同類別的失敗，且無法捕捉其他方法所捕捉的問題。

5.1 分類學

單一架構無法涵蓋所有維度。生產部署至少需要預先授權與沙箱執行；基於模型的篩選則在威脅模型包含對抗性意圖分類的情況下，增加纵深防禦。

關於分類法的說明。封閉執行系統（如 ceLLMate [32] 和 Nemo-Claw [10] ）確實能在資源/網路層面防止未經授權的操作（例如，阻擋發往特定主機的出站流量）。其區別在於，它們無法強制執行需要了解組織政策的語義業務規則（如支出限制、收件人白名單）。基於模型的篩選若採用確定性解碼（溫度 =0）可能會產生可重複的輸出，但篩選模型的決策邊界是學習得出的，並可能受到對抗輸入的影響，這使其與靜態政策有所區別。

5.2 Complementarity

These architectures are complementary, not competing. Consider a concrete scenario: a prompt-injected agent attempts to read `/etc/passwd` via a `data.file.read` tool call.

- **Pre-action authorization (OAP):** The passport's `allowed_classifications` does not include system files. Decision: DENY with `oap.capability_missing`. The read never executes.
- **Sandboxed execution:** The sandbox's filesystem restrictions block access to `/etc/passwd`. The read attempt executes but is contained.
- **Model-based screening:** The screening model may or may not flag the intent as malicious, depending on how the prompt injection is framed.

Now consider a different scenario: the agent makes a legitimate-looking API call (`payments.charge`, $500, USD) within the sandbox's network permissions but exceeding the agent's $100 per-transaction limit.

- **Pre-action authorization:** Decision: DENY with `oap.limit_exceeded`. The charge never executes.
- **Sandboxed execution:** The sandbox allows the network call. The $500 charge executes.
- **Model-based screening:** The screening model sees a valid API call pattern. No flag raised.

Microsoft's Defender for AI Agents [17] exemplifies the combined approach: a pre-action security evaluator analyzes intent before execution, and actions then run in a controlled environment. This layered architecture (pre-action authorization + sandboxed execution) is the minimum for production agents that handle money, data, or code.

6 Evaluation

6.1 Adversarial Testbed: APort Vault CTF

APort operates **Vault** (vault.aport.io), a live adversarial testbed in which participants attempt to manipulate an AI banking agent into making unauthorized transfers through social engineering. The agent uses a frontier language model for conversation and an OAP policy engine for authorization. The model's judgment can be compromised through conversation; the policy engine evaluates deterministically from the signed passport.

Results (as of March 2026):

Metric	Value
Total sessions	1,151
Total authorization decisions	4,437
Decisions denied	2,419 (54.5%)
Top denial: `oap.unknown_capability`	1,453
Top denial: `oap.merchant_forbidden`	412
Top denial: `oap.evaluation_error`	173 (see note)
Tier 5 ("The Vault") attempts	879
Tier 5 successful breaches	0
Tier 5 unclaimed prize	$5,000

Table 5: APort Vault CTF results.

Key finding: In multiple sessions, participants successfully convinced the AI model that a transfer was authorized, legitimate, and urgent. The model generated the tool call. The OAP policy engine denied it because the amount exceeded `currency_limits.USD.max_per_tx` or the recipient was not in `allowed_recipients`. The model's judgment was overridden through adversarial conversation; the deterministic policy evaluation was not.

5.2 互補性

這些架構是互補而非競爭。考慮一個具體情境：一個提示注入的代理嘗試透過 `data.file.read`工具呼叫來讀取 `/etc/passwd` 。

- 預行動授權 (OAP)：護照的 `allowed_classifications` 不包含系統檔案。決定：拒絕，原因為 `oap.capability_missing`。讀取操作從未執行。
- 封閉式執行：沙箱的檔案系統限制阻擋了對 `/etc/passwd`的存取。讀取嘗試已執行但被限制。
- 基於模型的篩選：篩選模型可能會或不會將意圖標記為惡意，這取決於提示注入的表述方式。

現在考慮另一種情境：代理在沙盒的網路權限範圍內做出看似合法的 API 呼叫（`payments.charge`，$500，美元），但超過了代理每筆交易$100的限額。

- 預行動授權：決定：拒絕，原因為 `oap.limit_exceeded`。該扣款從未執行。
- 封閉式執行：封閉式環境允許網路呼叫。$500 的費用將執行。
- 基於模型的篩選：篩選模型識別到有效的 API 呼叫模式。未觸發任何標記。

Microsoft 的 Defender for AI Agents [17] 展示了這種結合方法的範例：執行前的安全評估器會分析意圖，然後在受控環境中執行動作。這種分層架構（執行前授權 +沙盒執行）是處理金錢、資料或程式碼的生產環境代理的最小要求。

6 評估

6.1 對抗測試環境：APort Vault CTF

APort 運營 Vault（vault.aport.io），這是一個實時對抗測試環境，參與者試圖透過社會工程學手段操縱 AI 銀行代理進行未經授權的轉帳。該代理使用前沿語言模型進行對話，並使用 OAP 策略引擎進行授權。模型的判斷可能透過對話受到影響；策略引擎則從簽名護照中進行確定性評估。

結果（截至 2026 年 3 月）：

指標	值
總計會話數	1,151
總授權決定	4,437
駁回決定	2,419（54.5%）
最高否認：`oap.unknown_capability`	1,453
最高否認：`oap.merchant_forbidden`	412
最高否認：`oap.evaluation_error`	173（見註解）
第 5 級（「保險庫」）嘗試	879
五級成功滲透	0
第 5 級未領獎品	$5,000

表 5：APort Vault CTF 結果。

關鍵發現：在多次會話中，參與者成功說服 AI 模型認為轉帳已獲授權、合法且緊急。模型生成了工具呼叫。OAP 策略引擎因金額超過 `currency_limits.USD.max_per_tx` 或受款人不在 `allowed_recipients`中而拒絕。模型判斷透過對抗性對話被覆寫；但確定性策略評估則未被覆寫。

Built-in baseline: Tier 1 vs. Tier 5. The Vault's five tiers (T1–T5, distinct from OAP assurance levels L0–L4FIN) provide a natural controlled comparison. Tier 1 ("The Intern") uses the same model, interface, and OAP infrastructure but with a permissive policy (wildcard merchants, high limits). Tier 5 ("The Vault") uses a restrictive policy (zero capabilities, $0 limit). Against a comparable population of attackers:

Condition	Att.	OAP Dec.	Wins	Succ.	Block
T1: Permissive	788	586	588	74.6%	1%
T5: Restrictive	879	1,911	0	0%	100%

Table 6: T1 vs. T5 baseline comparison. "Att." = attempts, "OAP Dec." = OAP policy decisions, "Succ." = success rate, "Block" = OAP block rate.

At T1, 202 of 788 attempts (25.6%) never triggered an OAP decision: the model declined to generate a `transfer_funds` tool call, representing the model alignment layer filtering requests before they reached the policy engine. Of the 586 that reached OAP, 581 were allowed and 5 were denied. (Seven T1 wins predate full OAP integration during the pre-launch beta period, accounting for the 588 vs. 581 discrepancy.) At T5, every attempt that reached the policy engine was denied. The policy layer, not the model, determined the outcome: when policy permits most actions (T1), 74.6% of social engineering attempts succeed; when policy enforces strict capability scoping (T5), 0% succeed.

Note on `oap.evaluation_error`. The 173 `evaluation_error` denials (7.1% of all denials) represent malformed tool call contexts: missing required fields, incorrect parameter types, or incomplete context objects. These are input validation failures at the policy evaluation boundary, not non-determinism in the evaluation engine. The engine deterministically returns DENY with `oap.evaluation_error` when it cannot evaluate incomplete input. All 173 occurred at T5 (150) and T1–T4 (23), consistent with adversarial participants sending deliberately malformed requests.

Threats to validity. (1) Vault participants self-select and skew toward social engineering rather than protocol-level attacks; results may not generalize to APT-grade adversaries. (2) The T1/T5 comparison is observational, not a randomized controlled trial: participants chose which tiers to attempt, and T5 attracted more persistent attackers (median 2 turns at T1 vs. 2 turns at T5, but with a longer tail of multi-turn attempts). (3) The Vault tests a single domain (banking); generalization to code execution or multi-agent delegation is not demonstrated. (4) The T5 policy permits only transfers within the passport's `allowed_recipients` and `currency_limits`; participants did not have access to the policy specification. A companion empirical paper with the full CTF methodology, attack taxonomy, and statistical analysis is forthcoming.

6.2 Performance Benchmarks

Measured on production workloads (N=1,000 requests per mode):

Mode	p50	p95	p99	N
Cloud API (agent_id, path)	53 ms	63 ms	76 ms	1,000
Cloud API (agent_id, body)	53 ms	62 ms	77 ms	1,000
Cloud API (passport, path)	54 ms	63 ms	74 ms	1,000
Cloud API (passport, body)	53 ms	63 ms	71 ms	1,000
Local evaluation	174 ms	243 ms	358 ms	1,000

Table 7: OAP authorization latency benchmarks.

Cloud API median is 53 ms (server-side processing at Cloudflare edge; excludes client-side

內建基準：Tier 1 與 Tier 5 的比較。Vault 的五個等級（T1–T5，與 OAP 保證等級 L0–L4FIN 不同）提供了自然的受控比較。Tier 1（「實習生」）使用相同的模型、介面和 OAP 基礎設施，但採用寬鬆策略（通配符商家、高額度限制）。Tier 5（「Vault」）採用嚴格策略（零功能、$0 額度限制）。針對可比較的攻擊者群體：

狀況	Att.	OAP 12 月	Wins	成功。	區塊
T1：寬鬆	788	586	588	74.6%	1%
T5：限制	879	1,911	0	0%	100%

表 6：T1 與 T5 的基準比較。「Att.」 =嘗試次數，「OAP Dec.」 = OAP 策略決策，“成功率” = ，“阻塞” =OAP 阻塞率。

在 T1 階段，788 次嘗試中有 202 次（25.6%）從未觸發 OAP 決策：模型拒絕生成 `transfer_funds` 工具調用，代表模型對齊層在請求進入策略引擎前進行過濾。在 586 次進入 OAP 的嘗試中，581 次獲准，5 次被拒。（七次 T1 勝利發生在 OAP 完全整合之前的預發版測試期，這解釋了 588 與 581 的差異。）在 T5 階段，所有進入策略引擎的嘗試均被拒。是策略層而非模型決定了結果：當策略允許大多數操作時（T1），74.6% 的社會工程嘗試成功；當策略強制嚴格的能力範圍限制時（T5），成功率為 0%。

關於 `oap.evaluation_error` 的說明。 173 `evaluation_error` 項拒批（佔所有拒批的 7.1%）代表工具呼叫上下文格式錯誤：缺少必要欄位、參數類型不正確，或上下文物件不完整。這些是政策評估邊界處的輸入驗證失敗，而非評估引擎的非確定性行為。當引擎無法評估不完整輸入時，會確定性地返回 DENY，且 `oap.evaluation_error` 會出現在該情況下。所有 173 筆案例均發生於 T5（150 筆）與 T1 至 T4（23 筆），這與惡意參與者故意發送格式錯誤請求的情況相符。

有效性威脅。(1) 倉庫參與者自我選擇，傾向於社會工程攻擊而非協議層攻擊；結果可能無法推廣至 APT 級對手。(2) T1/T5 比較為觀察性研究，非隨機對照試驗：參與者自行選擇嘗試哪些層級，且 T5 吸引了更多持續攻擊者（T1 中位數為 2 次嘗試，T5 為 2 次嘗試，但 T5 有多次嘗試的長尾分佈）。(3) 倉庫測試單一領域（銀行）；未證明可推廣至程式執行或多代理委託。(4) T5 策略僅允許在護照的 `allowed_recipients` 和 `currency_limits`之間進行轉帳；參與者無法取得策略規範。附有完整 CTF 方法論、攻擊分類法與統計分析的隨文實證論文即將發表。

6.2 效能基準測試

基於生產工作負載（N=1,000 次請求/模式）測量：

Mode	p50	p95	p99	N
雲端 API (agent_id, path)	53 毫秒	63 毫秒	76 毫秒	1,000
Cloud API (agent_id, body)	53 毫秒	62 毫秒	77 毫秒	1,000
雲端 API（passport、path）	54 毫秒	63 毫秒	74 毫秒	1,000
雲端 API（passport、body）	53 毫秒	63 毫秒	71 毫秒	1,000
本地評估	174 毫秒	243 毫秒	358 毫秒	1,000

表 7：OAP 授權延遲基準測試。

Cloud API 中位數為 53 ms（在 Cloudflare 邊緣進行伺服器端處理；不包含客戶端）

network round-trip). All four API configurations show consistent p50 (53–54 ms) and p99 under 77 ms. Local evaluation (no network) has a median of 174 ms due to Python subprocess overhead. These latencies are acceptable for agent tool calls, which are themselves I/O-bound (tool execution typically ranges from 200 ms to several seconds).

Breakdown (cloud API, approximate; some steps execute in parallel):

Component	Time
Passport lookup + cache	20 ms
Policy evaluation	15 ms
Decision signing (Ed25519)	10 ms
HTTP overhead (TLS, parsing, serialization)	∼8 ms
Synchronous total (blocking)	**∼53 ms**

Table 8: Cloud API latency breakdown.

Audit log write (∼9 ms) executes asynchronously after the decision is returned and does not contribute to measured latency.

6.3 Coverage Analysis: OAP vs. OWASP Agentic Top 10

OWASP Risk	Coverage	Mechanism
Agent Goal Hijack	Partial	Policy denies actions outside declared capabilities
Tool Misuse	Partial	Per-tool capability check + parameter constraints
Privilege Escalation	Partial	Capability-scoped passport blocks cross-capability escalation
Identity & Privilege Abuse	Full	Assurance-level gating; Ed25519 verification
Improper Output Handling	None	Out of scope (model output, not action)
Excessive Agency	Partial	`max_calls_per_minute`, `approval_required`
Agentic Supply Chain	Partial	`allowed_servers` for MCP; `allowed_domains` for web
Knowledge Poisoning	None	Out of scope (content, not action)
Logging & Monitoring	Partial	Signed audit log; no anomaly detection
Uncontrolled Autonomy	Partial	Fail-closed; ESCALATE not yet implemented

Table 9: OAP coverage of OWASP Agentic Top 10 risks.

OAP provides full or partial coverage for 8 of 10 OWASP Agentic risks. The two uncovered risks (Improper Output Handling, Knowledge Poisoning) operate at the content layer, orthogonal to action-authorization.

網路往返時間）。所有四種 API 配置均顯示一致的 p50（53–54 毫秒）及 p99 低於 77 毫秒。本地評估（無網路）因 Python 子進程開銷，中位數為 174 毫秒。這些延遲對於代理工具呼叫是可接受的，因為工具呼叫本身即為 I/O 密集型（工具執行通常介於 200 毫秒至數秒之間）。

分解（雲端 API，約數；部分步驟會並行執行）：

組件	Time
護照查詢 + 快取	20 毫秒
政策評估	15 毫秒
簽章決策 (Ed25519)	10 毫秒
HTTP 開銷（TLS、解析、序列化）	∼8 毫秒
同步總計（阻塞）	**∼53 毫秒**

表 8：雲端 API 延遲分析。

審計日誌寫入 (∼9ms) 會在決策返回後異步執行，且不計入測量延遲。

6.3 覆蓋分析：OAP 與 OWASP 代理十大威脅

OWASP 風險	覆蓋範圍	機制
代理目標劫持	部分	策略會拒絕超出已宣告能力的動作
工具濫用	部分	工具能力檢查 + 參數限制
權限提升	部分	以能力為範圍的憑證可防止跨能力權限提升
身分與特權濫用	Full	保證級閘門；Ed25519 驗證
不正確的輸出處理	None	不在範圍內（模型輸出，非行動）
過度的代理權	部分	`max_calls_per_minute`, `approval_required`
代理供應鏈	部分	`allowed_servers` 用於 MCP； `allowed_domains` 用於網頁
知識投毒	None	不在範圍內（內容，非行為）
日誌記錄與監控	部分	已簽署的審計日誌；無異常檢測
不受控的自主性	部分	失敗即關閉；ESCALATE 尚未實作

表 9：OWASP 代理 Top 10 風險的 OAP 覆蓋率。

OAP 涵蓋了 10 項 OWASP 代理風險中的 8 項。未涵蓋的兩項風險（不當輸出處理、知識投毒）位於內容層，與行動授權正交。

6.4 Implementation Status

Component	Detail
Specification	OAP v1.0, DOI: 10.5281/zenodo.18901596
License	Apache 2.0
Core package	`@aporthq/aport-agent-guardrails` v1.0.15
Framework adapters	7 (5 production + 1 beta + 1 proposed)
Policy packs	21 published
Signing algorithm	Ed25519 (PKCS8 encoding)
Hash chain	SHA-256, per-agent, write-once KV
Passport registry	Cloudflare Workers, multi-region
Service discovery	`.well-known/oap/` (RFC 8615)

Table 10: OAP implementation status.

7 Standards Alignment

OAP aligns with NIST AI RMF [29] functions (GOVERN, MAP, MEASURE, MANAGE) and maps to five NIST SP 800-53 control families (AC, AU, IA, SC, SI). Detailed mappings are in Appendix B. The more significant standards contribution is the proposal of a new control category:

7.1 Proposed Control Category: Pre-Action Authorization (PAA)

We propose that standards bodies recognize pre-action authorization as a distinct control category:

- **PAA-1**: The organization SHALL define a machine-readable authorization policy specifying which tool calls an AI agent is permitted to execute, under what conditions, at what assurance level.
- **PAA-2**: The organization SHALL implement a platform-level hook that enforces the policy synchronously before each tool call, independent of the model's reasoning.
- **PAA-3**: The organization SHALL issue verifiable credentials (agent passports) binding agents to authorized capability scopes.
- **PAA-4**: The organization SHALL maintain a tamper-evident audit log of all authorization decisions.
- **PAA-5**: In the absence of a valid authorization decision, tool calls SHALL be denied by default.

PCAS [15] and this work independently converge on the need for deterministic pre-action enforcement as a distinct architectural layer, differing primarily in policy language and deployment model. This convergence suggests the category is real, not an artifact of a single system's design.

8 Discussion

8.1 Limitations

Delegation chains. OAP v1.0 does not formalize a delegation model for multi-agent scenarios. An agent that delegates to a sub-agent with narrowed permissions requires a delegation chain specification. This is planned for v1.1.

Policy expressiveness. OAP policies use declarative JSON/YAML, simpler to author and audit but less expressive than PCAS's Datalog-derived language [15]. Complex conditional logic (e.g., "allow if the caller's manager approved AND the target account is in the same jurisdiction

6.4 實施狀況

組件	詳細
規格	OAP v1.0，DOI: 10.5281/zenodo.18901596
授權條款	Apache 2.0
核心套件	`@aporthq/aport-agent-guardrails` v1.0.15
框架適配器	7 (5 生產 + 1 beta + 1 建議)
策略包	已發布 21 筆
簽名演算法	Ed25519 (PKCS8 編碼)
Hash chain	SHA-256，按代理，寫入一次 KV
護照登記簿	Cloudflare Workers，多區域
服務發現	`.well-known/oap/` (RFC 8615)

表 10：OAP 實施狀況。

7 標準對齊

OAP 與 NIST AI RMF [29] 功能（GOVERN、MAP、MEASURE、MANAGE）一致，並對應五個 NIST SP 800-53 控制家族（AC、AU、IA、SC、SI）。詳細對應關係見附錄 B。更具體的標準貢獻在於提出新的控制類別：

7.1 建議控制類別：預先授權（PAA）

我們建議標準制定機構將事前授權視為獨立的控制類別：

- PAA-1：組織必須定義一臺可讀取機器的授權政策，說明在什麼條件下、於何種保證等級下，允許哪些工具呼叫 AI 代理執行。
- PAA-2：組織必須在每次工具呼叫之前，獨立於模型的推理，同步實施一級平台鉤子以強制執行策略。
- PAA-3：組織必須發行可驗證憑證（代理護照），將代理綁定至授權能力範圍。
- PAA-4：組織必須維護一份所有授權決策的防篡改審計日誌。
- PAA-5：若無有效的授權決定，工具呼叫應預設予以拒絕。

PCAS [15] 與本工作獨立地都認為需要確定的預動作
作為一個獨立的架構層，其差異主要在於政策語言與部署模式。這種融合表明該類別是真實存在的，而非單一系統設計的產物。

8 討論

8.1 限制

代理鏈。OAP v1.0 尚未為多代理場景正式化授權模型。若代理將權限縮減後授權給子代理，則需指定授權鏈。此功能規劃於 v1.1 版本實現。

策略表達力。OAP 策略採用宣告式 JSON/YAML，較易撰寫與審計，但表達力不如 PCAS 的基於 Datalog 導出的語言 [15]。複雜的條件邏輯（例如：「若呼叫者的主管已核准且目標帳戶位於同一司法管轄區，則允許」

AND the amount is below the 30-day rolling average") requires either policy composition or extension of the rule language.

Adaptive policies. OAP policies are static; they must be authored and updated manually. AgentGuardian [16] demonstrates that policies learned from execution traces can adapt to novel attack patterns. A hybrid approach (static policy baseline + learned anomaly detection) is a natural extension.

Scope boundary. OAP enforces authorization at tool call boundaries. Actions that do not pass through a tool calling interface (direct model output rendering, in-context retrieval, side-channel communication) are out of scope. Sandboxed execution [10, 11] provides the complementary containment layer for these cases.

Composability attacks. OAP evaluates each tool call independently. A sequence of individually-permitted calls could collectively achieve an unauthorized outcome (e.g., multiple small transfers that exceed an aggregate limit—a "structuring" attack). The OAP v1.1 draft includes sliding-window policy packs that maintain per-agent aggregate state to detect and block sequence-based structuring, alongside delegation chain formalization.

ESCALATE completeness. The ESCALATE decision path (Algorithm 1, line 14) is specified but not yet implemented in the reference. Deployments requiring human-in-the-loop approval currently implement ESCALATE at the application layer.

Scale. Current benchmarks are at modest scale (∼1,000 concurrent evaluations). Performance at 10,000+ evaluations per second, typical of large enterprise deployments, requires dedicated benchmarking.

Platform trust. OAP assumes the framework runtime is not compromised (Section 4.1). If an adversary controls the runtime, the hook can be bypassed entirely. Integration with TEE-based attestation [34] could reduce this assumption.

8.2 The Full Safety Stack

A production agent safety architecture requires at minimum four layers: (1) model alignment, which shapes behavior probabilistically at training time; (2) pre-action authorization, which blocks unauthorized tool calls deterministically before execution; (3) sandboxed execution, which contains the blast radius of actions that pass authorization; and (4) post-hoc evaluation, which finds systemic issues across completed runs and feeds improvements back into alignment and policy.

Remove any one and there is a structural gap. Alignment without authorization is probabilistic enforcement. Authorization without sandboxing lacks blast-radius containment. Sandboxing without authorization permits semantically unauthorized actions within permitted resource scope. Evaluation without runtime enforcement finds problems only after damage is done.

8.3 From Security to Accountability

The same infrastructure that enforces security constraints (identity, policy, cryptographic proof) serves a broader accountability function. Policy packs can encode quality gates (minimum test coverage before deployment), operational contracts (SLA requirements before task delegation), and compliance controls (data residency before cross-border transfer). The agent passport is less an access control token and more an accountability credential: a portable record of what an agent is authorized, required, and expected to do.

9 Conclusion

Autonomous AI agents executing tool calls without standardized authorization is the access control gap of the current era, analogous to the absence of RBAC in early multi-user systems and the absence of OAuth in early web APIs. The gap is real: 27.2% of engineering teams building

AND 金額低於 30 天滾動平均值）需要政策組合或規則語言的擴充。

適配性策略。OAP 策略是靜態的；必須手動撰寫和更新。AgentGuardian [16] 證明從執行軌跡中學習的策略可以適應新型攻擊模式。混合方法（靜態策略基線 + 學習異常檢測）是自然的延伸。

作用範圍邊界。OAP 在工具調用邊界強制執行授權。未通過工具調用介面（直接模型輸出渲染、上下文檢索、旁路通訊）執行的動作不在範圍內。沙盒執行 [10, 11] 為這些情況提供補充的封閉層。

可組合性攻擊。OAP 獨立評估每個工具調用。一系列個別允許的調用可能共同達成未經授權的結果（例如，多次小額轉帳超過總額限制——即「結構化」攻擊）。OAP v1.1 草案包含滑動窗口策略包，用於維護每個代理的聚合狀態，以檢測並阻擋基於序列的結構化攻擊，同時還包括委託鏈的形式化。

ESCALATE 完整性。ESCALATE 決策路徑（演算法 1，第 14 行）已定義但尚未在參考實作中實現。目前需要人類在迴路批准的部署在應用層實作了 ESCALATE。

規模。目前的基準測試處於小規模（∼1,000 個並行評估）。若要達到每秒 10,000+ 個評估的性能（這通常是大型企業部署的典型水平），則需要進行專門的基準測試。

平台信任。OAP 假設框架運行時未被攻陷（第 4.1 節）。若攻擊者控制運行時，則可完全繞過掛鉤。與基於 TEE 的驗證 [34] 的整合可降低此假設。

8.2 完整安全堆疊

一個生產級代理安全架構至少需要四層：(1) 模型對齊，在訓練階段以概率方式塑造行為；(2) 行動前授權，在執行前以確定性方式阻擋未經授權的工具調用；(3) 沙盒執行，將通過授權的行動的影響範圍限制在內；以及 (4) 事後評估，在已完成運行中發現系統性問題，並將改進反饋至對齊與策略中。

移除任何一項，就會產生結構性缺口。未經授權的權限分配是概率性執行。未經沙箱隔離的授權缺乏爆炸半徑的封閉。未經授權的沙箱隔離允許在許可資源範圍內執行語義上未經授權的操作。未經執行時強制評估的評估，僅在造成損害後才能發現問題。

8.3 從安全到負責

執行安全限制（身分、策略、加密證明）的相同基礎設施，也承擔更廣泛的責任歸屬功能。策略套件可編碼品質閘（部署前的最低測試覆蓋率）、營運合約（任務委託前的服務等級協議要求）以及合規控制（跨境傳輸前的資料駐留）。代理護照不僅僅是存取控制令牌，更是一種責任歸屬憑證：一份可攜式的記錄，說明代理被授權、被要求且預期執行的事項。

9 結論

自主 AI 代理在不具備標準化授權的情況下執行工具調用，是當前時代的存取控制缺口，這相當於早期多使用者系統缺乏 RBAC，以及早期網路 API 缺乏 OAuth。此缺口確實存在：27.2% 的開發團隊在構建

custom authorization from scratch [2], 492+ MCP servers deployed without authentication [3], 824+ malicious skills in the OpenClaw ecosystem [35], and autonomous agents leaking SSNs through single-verb reframing [1]. In a live adversarial testbed (4,437 decisions across 1,151 sessions), social engineering succeeded 74.6% of the time when the model was the primary defense; deterministic policy enforcement reduced this to 0% (879 highest-tier attempts, $5,000 prize unclaimed).

Pre-action authorization is distinct from alignment, evaluation, and sandboxing. It complements all three.

OAP is an open-source, specification-first implementation of this layer, deployed across six agent frameworks (with one proposed) and evaluated against live adversarial attacks. Limitations remain: delegation chains are not yet formalized, policy expressiveness is bounded by declarative JSON, benchmarks are at modest scale (∼1,000 concurrent evaluations; enterprise-scale validation at 10,000+ evaluations/second is needed), and the ESCALATE path is specified but not yet implemented. The specification is released under Apache 2.0 (DOI: 10.5281/zenodo.18901596).

Disclosure

The author is the founder of APort Technologies Inc., which develops and operates the OAP reference implementation evaluated in this paper. The specification [30] is released under Apache 2.0, all artifacts are publicly available, and any organization can build a conforming implementation. OAP's OpenClaw integration (Section 3) predates the Cisco ClawHavoc research [35]; the Cisco study was conducted independently and no commercial relationship exists between APort and OpenClaw. The Vault CTF dataset will be released (anonymized) alongside the companion empirical paper.

References

[1] N. Shapira, C. Wendler, A. Yen, G. Sarti, K. Pal, O. Floody, et al. Agents of chaos. *arXiv:2602.20021*, February 2026.

[2] Gravitee. State of AI agent security 2026: Authorization patterns in production agentic deployments. March 2026.

[3] Invariant Labs. MCP security audit: 492+ MCP servers exposed without authentication. March 2026.

[4] CVE-2026-26118. Azure MCP server SSRF. CVSS 8.8. NVD, 2026.

[5] Kiteworks. Agents of chaos: Why 90% of organizations lack purpose-binding controls. March 2026.

[6] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv:2302.12173*, 2023.

[7] Q. Zhan et al. Removing RLHF protections in GPT-4 via fine-tuning. *arXiv:2311.05553*, 2023.

[8] OpenAI. OpenAI to acquire Promptfoo. openai.com/index, March 2026.

[9] Galileo AI. Open-source agent evaluation framework. March 2026.

[10] NVIDIA. NemoClaw: Enterprise security for OpenClaw agents. GTC 2026.

[11] E2B. Secure sandboxes for AI agents. e2b.dev.

從零開始自訂授權 [2], 492+ 未部署身份驗證的 MCP 伺服器 [3],824+ OpenClaw 生態系中的惡意技能 [35], 以及透過單詞重構洩漏社會安全號碼的自主代理 [1]。在一個實戰對抗測試環境中（涵蓋 1,151 場會話、共 4,437 次決策），當模型是主要防禦手段時，社會工程學攻擊成功率達 74.6%；而執行確定性政策後，此成功率降至 0%（879 次最高級別嘗試，5,000 美元獎金未領）。

先驗授權與對齊、評估及沙盒測試不同。它與這三者相輔相成。

OAP 是一個開源、以規範為優先的該層實現，已部署於六個代理框架（其中一個為提案），並針對真實對抗攻擊進行評估。現存限制包括：委託鏈尚未形式化，策略表達能力受限於宣告式 JSON，基準測試規模尚屬中等（∼1,000 並行評估；企業級驗證需達 10,000+ 評估/秒），且 ESCALATE 路徑已定義但尚未實現。本規範採用 Apache 2.0 授權（DOI: 10.5281/zenodo.18901596）。

揭露

作者為 APort Technologies Inc. 創辦人，該公司開發並營運本論文所評估的 OAP 參考實作。規範 [30] 採用 Apache 2.0 授權發布，所有資源均公開可取，任何組織皆可建置符合規範的實作。OAP 的 OpenClaw 整合功能（第 3 節）早於 Cisco ClawHavoc 研究 [35]；Cisco 研究為獨立進行，APort 與 OpenClaw 之間不存在商業關係。Vault CTF 數據集將與隨附的實證論文一同發布（經匿名化處理）。

參考資料

[1] N. Shapira, C. Wendler, A. Yen, G. Sarti, K. Pal, O. Floody, et al. Agents of chaos. *arXiv:2602.20021*, February 2026.

[2] Gravitee. State of AI agent security 2026: Authorization patterns in production agentic deployments. March 2026.

[3] Invariant Labs. MCP security audit: 492+ MCP servers exposed without authentication. March 2026.

[4] CVE-2026-26118. Azure MCP server SSRF. CVSS 8.8. NVD, 2026.

[5] Kiteworks. Agents of chaos: Why 90% of organizations lack purpose-binding controls. March 2026.

[6] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv:2302.12173*, 2023.

[7] Q. Zhan et al. Removing RLHF protections in GPT-4 via fine-tuning. *arXiv:2311.05553*, 2023.

[8] OpenAI. OpenAI to acquire Promptfoo. openai.com/index, March 2026.

[9] Galileo AI. Open-source agent evaluation framework. March 2026.

[10] NVIDIA. NemoClaw: Enterprise security for OpenClaw agents. GTC 2026.

[11] E2B. Secure sandboxes for AI agents. e2b.dev.

[12] Modal. Serverless cloud for AI agent execution. modal.com.

[13] Google. Agent Development Kit (ADK): Safety and security. google.github.io/adk-docs/safety.

[14] NVIDIA. Practical security guidance for sandboxing agentic workflows. developer.nvidia.com, 2026.

[15] N. Palumbo, S. Choudhary, J. Choi, P. Chalasani, and S. Jha. Policy compiler for secure agentic systems. *arXiv:2602.16708*, 2026.

[16] N. Abaev et al. AgentGuardian: Learning access control policies to govern AI agent behavior. *arXiv:2601.10440*, 2026.

[17] Microsoft. From runtime risk to real-time defense: Securing AI agents. Microsoft Security Blog, January 2026.

[18] M. Bhattarai and M. Vu. Trustworthy agentic AI requires deterministic architectural boundaries. *arXiv:2602.09947*, 2026.

[19] D. Hardt et al. RFC 6749: The OAuth 2.0 authorization framework. IETF, 2012.

[20] Linux Foundation. Agentic AI Infrastructure Foundation (AAIF): Founding charter. 2026.

[21] OWASP. Top 10 for agentic applications 2026. genai.owasp.org.

[22] OWASP. MCP Top 10. owasp.org/www-project-mcp-top-10.

[23] SAFE-MCP. Security baseline for MCP-enabled systems. Adopted by Linux Foundation and OpenID Foundation, 2026.

[24] NIST. AI agent standards initiative. nist.gov, February 17, 2026.

[25] Google / Linux Foundation. Agent2Agent (A2A) Protocol v1.0.0. a2a-protocol.org.

[26] A2A Project. Secure passport extension. github.com/a2aproject/a2a-samples.

[27] U. Uchibeke. A2A Discussion #1404: Pre-action authorization via OAP. github.com/a2aproject/A2A/discussions/1404.

[28] M. Nottingham and E. Hammer-Lahav. RFC 8615: Well-known uniform resource identifiers. IETF, 2019.

[29] NIST. AI Risk Management Framework (AI RMF) 1.0. 2023.

[30] U. Uchibeke. Open Agent Passport (OAP) v1.0 specification. Zenodo. DOI: 10.5281/zenodo.18901596, 2026.

[31] The attack and defense landscape of agentic AI: A comprehensive survey. *arXiv:2603.11088*, March 2026.

[32] L. Meng, E. Fernandes, et al. ceLLMate: Sandboxing browser AI agents. *arXiv:2512.12594v2*, March 2026.

[33] Y. Huang, P. Sattigeri, X. Zhang, et al. Building a foundational guardrail for general agentic systems via synthetic data. *arXiv:2510.09781*, October 2025.

[34] X. Jin, X. Ren, et al. Proof-of-guardrail in AI agents and what (not) to trust from it. *arXiv:2603.05786*, March 2026.

[35] Cisco AI Threat and Security Research Team. Personal AI agents like OpenClaw are a security nightmare. Cisco Blogs, January 30, 2026.

[12] Modal. Serverless cloud for AI agent execution. modal.com.

[13] Google. Agent Development Kit (ADK): Safety and security. google.github.io/adk-docs/safety.

[14] NVIDIA. Practical security guidance for sandboxing agentic workflows. developer.nvidia.com, 2026.

[15] N. Palumbo, S. Choudhary, J. Choi, P. Chalasani, and S. Jha. Policy compiler for secure agentic systems. *arXiv:2602.16708*, 2026.

[16] N. Abaev et al. AgentGuardian: Learning access control policies to govern AI agent behavior. *arXiv:2601.10440*, 2026.

[17] Microsoft. From runtime risk to real-time defense: Securing AI agents. Microsoft Security Blog, January 2026.

[18] M. Bhattarai and M. Vu. Trustworthy agentic AI requires deterministic architectural boundaries. *arXiv:2602.09947*, 2026.

[19] D. Hardt et al. RFC 6749: The OAuth 2.0 authorization framework. IETF, 2012.

[20] Linux Foundation. Agentic AI Infrastructure Foundation (AAIF): Founding charter. 2026.

[21] OWASP. Top 10 for agentic applications 2026. genai.owasp.org.

[22] OWASP. MCP Top 10. owasp.org/www-project-mcp-top-10.

[23] SAFE-MCP. Security baseline for MCP-enabled systems. Adopted by Linux Foundation and OpenID Foundation, 2026.

[24] NIST. AI agent standards initiative. nist.gov, February 17, 2026.

[25] Google / Linux Foundation. Agent2Agent (A2A) Protocol v1.0.0. a2a-protocol.org.

[26] A2A Project. Secure passport extension. github.com/a2aproject/a2a-samples.

[27] U. Uchibeke. A2A Discussion #1404: Pre-action authorization via OAP. github.com/a2aproject/A2A/discussions/1404.

[28] M. Nottingham and E. Hammer-Lahav. RFC 8615: Well-known uniform resource identifiers. IETF, 2019.

[29] NIST. AI Risk Management Framework (AI RMF) 1.0. 2023.

[30] U. Uchibeke. Open Agent Passport (OAP) v1.0 specification. Zenodo. DOI: 10.5281/zenodo.18901596, 2026.

[31] The attack and defense landscape of agentic AI: A comprehensive survey. *arXiv:2603.11088*, March 2026.

[32] L. Meng, E. Fernandes, et al. ceLLMate: Sandboxing browser AI agents. *arXiv:2512.12594v2*, March 2026.

[33] Y. Huang, P. Sattigeri, X. Zhang, et al. Building a foundational guardrail for general agentic systems via synthetic data. *arXiv:2510.09781*, October 2025.

[34] X. Jin, X. Ren, et al. Proof-of-guardrail in AI agents and what (not) to trust from it. *arXiv:2603.05786*, March 2026.

[35] Cisco AI Threat and Security Research Team. Personal AI agents like OpenClaw are a security nightmare. Cisco Blogs, January 30, 2026.

A Artifacts and Reproducibility

All artifacts required to reproduce the results in this paper are publicly available under Apache 2.0:

Artifact	Location	Purpose
OAP v1.0 Spec	DOI: 10.5281/zenodo.18901596	Canonical spec
Spec source	github.com/aporthq/aport-spec	Living spec
Policy packs	github.com/aporthq/aport-policies	Policy library
Reference impl.	github.com/aporthq/aport-agent-guardrails	Evaluator
Adversarial testbed	vault.aport.io	Live CTF
API endpoint	aport.io / aport.id	Registry
API docs	docs.aport.io	Integration
NIST RFI	NIST-2025-0035 (March 2026)	Standards
A2A proposal	github.com/a2aproject/A2A/discussions/1404	Protocol

Table 11: Specification artifacts.

To reproduce performance benchmarks:

1. Install `@aporthq/aport-agent-guardrails` v1.0.15 from npm.
2. Create a passport via `npx aport-id` or at aport.id.
3. Run `POST /api/verify/policy/{capability_id}` with timing instrumentation (N=1,000).

To reproduce CTF results: live results are available at vault.aport.io; the anonymized dataset will be released with the companion empirical paper.

B NIST Alignment Detail

NIST AI RMF Mapping (function-level alignment; sub-category mapping requires organizational context):

AI RMF Function	OAP Contribution
GOVERN	Policy packs encode organizational authorization policy in machine-readable, auditable form
MAP	Assurance levels (L0–L4FIN) map agent capabilities to risk categories
MEASURE	Signed per-decision audit log enables denial rate, escalation rate, and coverage measurement
MANAGE	Policy packs updated without model retraining; passport suspension propagates within 30 s

Table 12: NIST AI RMF mapping.

NIST SP 800-53 Alignment:

A 人工制品與可重複性

本論文所需的所有重現結果的藝術品均公開提供，採用 Apache 2.0 授權：

神器	位置	目的
OAP v1.0 規範	DOI: 10.5281/zenodo.18901596	標準規範
規格來源	github.com/aporthq/aport-spec	現存樣本
策略包	github.com/aporthq/aport-policies	策略庫
參考實作。	github.com/aporthq/aport-agent-guardrails	評估者
對抗性測試床	vault.aport.io	現場 CTF
API 端點	aport.io / aport.id	註冊表
API 文檔	docs.aport.io	整合
NIST 詢答函	NIST-2025-0035（2026 年 3 月）	標準
A2A 提案	github.com/a2aproject/A2A/discussions/1404	協定

表 11：規範文檔。

要重現效能基準測試：

1. 從 npm 安裝 `@aporthq/aport-agent-guardrails` v1.0.15。
2. 透過 `npx aport-id` 或 aport.id 建立護照。
3. 執行 `POST /api/verify/policy/{capability_id}` 並加入計時儀器 (N=1,000)。若要重現 CTF 結果：即時結果可於 vault.aport.io 取得；匿名化數據集將隨同配套實證論文發布。

B NIST 對齊詳細資訊

NIST AI RMF 映射（功能級對齊；子類別映射需要組織背景）：

AI RMF 功能	OAP 貢獻
GOVERN	策略套件以機器可讀且可稽核的形式編碼組織授權策略
MAP	保障等級（L0–L4FIN）將代理能力映射至風險類別
測量	簽名逐決策審計日誌可啟用拒絕率、升級率及覆蓋率測量
管理	政策包更新無需重新訓練模型；護照暫停狀態在 30 秒內生效。

表 12：NIST AI RMF 映射。

NIST SP 800-53 對齊：

Control Family	OAP Mechanism
AC (Access Control)	Policy packs enforce least-privilege per tool type; passports are scoped credentials
AU (Audit)	Every tool call decision produces a signed log entry with decision, reason, and timestamp
IA (Identification)	Ed25519-signed passport provides cryptographic agent identity
SC (System Protection)	`.well-known/oap/` discovery over HTTPS; Ed25519 decision signatures
SI (System Integrity)	Fail-closed policy; hash-chained audit trail with write-once semantics

Table 13: NIST SP 800-53 alignment.

C Denial Response Handling

When OAP returns a DENY decision, the response includes:

```
{
  "decision": "DENY",
  "deny_code": "oap.limit_exceeded",
  "reason": "Amount $500 exceeds max_per_tx limit of $100",
  "decision_id": "dec_abc123...",
  "signature": "ed25519:..."
}
```

Agent frameworks use the structured denial to generate contextual responses rather than generic failures. For example, a banking agent receiving `oap.merchant_forbidden` can inform the user that the recipient is not in the approved list, without exposing internal policy details.

控制家族	OAP 機制
AC (存取控制)	策略套件針對工具類型強制執行最小權限原則；護照為範圍限定的憑證。
審計 (AU)	每次工具調用決策都會產生包含決策、原因與時間戳記的簽名日誌條目
IA（識別）	Ed25519 簽名的護照提供加密代理身份
SC（系統保護）	`.well-known/oap/` 透過 HTTPS 的發現；Ed25519 決策簽名
SI（系統完整性）	失敗即關閉策略；具有寫入一次語義的哈希鏈式審計日誌

表 13：NIST SP 800-53 對齊。

C 拒絕回應處理

當 OAP 返回 DENY 決定時，回應包含：

```
{
  "decision": "DENY",
  "deny_code": "oap.limit_exceeded",
  "reason": "Amount $500 exceeds max_per_tx limit of $100",
  "decision_id": "dec_abc123...",
  "signature": "ed25519:..."
}
```

代理框架利用結構化拒絕來生成情境化回應，而非通用錯誤。例如，銀行代理在收到 `oap.merchant_forbidden` 時，可告知用戶收款人不在核准名單中，而不洩露內部政策細節。